

Topic 2.7

2.7: Scope and Extent of Identifiers

The *scope* of an identifier indicates which routines and files can access the identifier. There are several scopes in C:

- Local to a function
- Visible only within the file declaring the identifier
- Global, and visible to all functions in all files

The *extent* of an identifier (variable) indicates how long a variable will live (retain its value). Several extents are possible:

- *Automatic*: The variable comes into existence when the function declaring it is invoked, and goes out of existence when the function returns.
- *Static*: The variable lives for the entire duration of the program.
- *Dynamic*: The programmer controls when the variable comes into existence and goes out of existence.

Here is a code fragment that illustrates scope and extent:

```
/*
 * FILENAME:      decls.c
 * PROGRAMMER:    Gary J. Blair
 * DATE:          7/21/1992
 * LANGUAGE:      ANSI C
 * PURPOSE:       A program skeleton to illustrate
 *                scope and extent.
 */

int y; /* This variable exists for the entire
        duration of the program, and may be
        accessed by any function in any file
        that is part of the program by declaring:
        extern int y;
        No such declaration is needed in this
        file. */

static int x; /* This variable exists for the
                entire duration of the program,
                but is only accessible to
                functions declared below it
                in the source file. static
                refers to scope. */

int r( int arg ) /* r may be accessed by any
                  function in any
                  file by declaring:
                  extern int r( int arg );
                  This declaration is not needed
                  in this file. */
{
    static int a; /* a exists for the duration of
```

```
the program, but is only
accessible from within the
function r. static refers to
extent. */

int b;      /* b is created each time r is
             called, and is destroyed each
             time b returns to the
             caller. b is only accessible
             from within the function r. */

auto int c; /* Same as b. */

register int d; /* Same as b, except compiler
                should try to keep d in a
                register for fast access. */
}

static int q( int arg ) /* q may only be accessed
                        by functions following
                        this declaration in this
                        source file. static
                        refers to scope. */
{ }
```

Topic 2.8 (<https://onlinelearning.berkeley.edu/courses/2291660/pages/topic-2-dot-8>)